

I'll inspect the workspace for the referenced PPT and any existing project files, then turn the stated workflow into an implementable development document covering architecture, Android constraints, blockchain storage, live call analysis, security, APIs, data models, testing, and delivery phases. # Trust Voice: Complete Development Document

1. Product Overview

****Trust Voice**** is an Android application that creates a verified voice identity for a user and analyzes live telephone conversations for suspected impersonation, fraud, or voice manipulation.

The proposed workflow is:

1. User registers in the application.
2. User records multiple voice samples.
3. The system creates a biometric voice profile.
4. The voice profile is encrypted and stored off-chain.
5. A cryptographic hash and ownership proof are recorded on a blockchain.
6. During a phone call, the application detects call state.
7. The user explicitly enables speakerphone and call monitoring.
8. The application captures microphone audio while the call is active.
9. Audio is analyzed live for voice identity, spoofing indicators, suspicious phrases, and anomalies.
10. The application shows risk alerts and stores an auditable result.

2. Important Technical Constraint

The proposed functionality cannot be implemented exactly as “automatically capture the complete normal phone-call audio” on every Android device.

Android restricts access to cellular call audio. Capturing `VOICE\_CALL`, uplink, or downlink audio generally requires privileged or system-level permissions unavailable to ordinary Play Store applications. Android's official documentation identifies call audio sources and notes that direct call capture requires restricted permissions.

([developer.android.com](https://developer.android.com/media/platform/sharing-audio-input?hl=en&utm_source=openai))

A practical implementation should therefore support these modes:

Mode A: Speakerphone Microphone Monitoring

The user:

1. Answers or starts the call.

2. Grants microphone permission.
3. Enables Trust Voice monitoring.
4. Puts the call on speakerphone.
5. Keeps the phone microphone available to the Trust Voice foreground service.

The application analyzes sound received through the device microphone. This is the closest implementation to the proposed workflow, but it has limitations:

- It may capture the user's voice and the remote speaker through acoustic leakage.
- Results depend on speaker volume and microphone placement.
- Bluetooth headsets may prevent the remote voice from reaching the phone microphone.
- Noise, echo, and room acoustics reduce accuracy.
- The application cannot guarantee access to both call channels.

Mode B: Trust Voice VoIP Calling

For reliable live analysis, Trust Voice should provide its own calling system using WebRTC, SIP, or a communications provider.

Audio is then available inside the application before being sent to the remote party. This provides:

- Reliable two-way audio access.
- Better echo cancellation.
- Lower latency.
- Consistent analysis.
- Better compliance and auditability.

Mode C: Enterprise/OEM Version

For managed devices, custom Android firmware, or a system-signed application, direct telephony audio capture may be possible. This is not the normal consumer Play Store model.

3. Recommended Product Strategy

Build the product in two stages:

MVP

- Voice enrollment.
- Encrypted voice profile creation.
- Blockchain ownership proof.
- Speakerphone-based monitoring.
- On-device or server-based live analysis.

- User-controlled monitoring.
- Visible recording/monitoring indicator.
- Post-call risk report.

Production Version

- Trust Voice VoIP calling.
- Two-channel audio analysis.
- Better fraud detection.
- Device integrity checks.
- Enterprise administration.
- Consent and compliance workflows.
- Multi-device voice identity synchronization.

The VoIP path should be treated as the primary long-term architecture. The normal phone-call speakerphone path should be treated as a compatibility feature with documented limitations.

4. Functional Requirements

4.1 User Registration

The user can:

- Create an account using email, phone, or passkey.
- Verify the account.
- Create a local device key.
- Accept privacy and biometric-processing agreements.
- View and revoke consent.

Recommended authentication:

- OAuth 2.1 or OpenID Connect.
- Short-lived access tokens.
- Refresh-token rotation.
- Optional passkey support.
- Device binding using Android Keystore.

4.2 Voice Enrollment

The application should guide the user through several recordings.

Recommended enrollment process:

1. Explain microphone and biometric data usage.

2. Ask the user to record 5 to 10 samples.
3. Use different phrases and speaking styles.
4. Require quiet-environment and normal-environment samples.
5. Check sample quality.
6. Remove silence and excessive noise.
7. Confirm that recordings belong to one speaker.
8. Generate a voice embedding.
9. Encrypt and store the voice profile.
10. Write the profile hash to the blockchain.

Example prompts:

- "My voice is my identity."
- "I am registering my voice with Trust Voice."
- Randomized phrases generated by the backend.
- Numeric and phonetic phrases to reduce replay attacks.

The application should not store raw recordings by default. Raw samples should be deleted after successful feature extraction unless the user explicitly enables retention.

4.3 Voice Profile

A voice profile should contain:

```
```text
user_id
profile_id
embedding_version
model_version
sample_count
quality_score
created_at
updated_at
encryption_key_reference
profile_hash
blockchain_transaction_id
status
```
```

The profile should contain an embedding rather than a raw voice recording whenever possible.

4.4 Call Detection

For standard cellular calls, use supported Android telecom integration.

Potential components:

- `TelecomManager`
- `InCallService`, where the application is configured as the default phone application
- Call state APIs
- Foreground service for active monitoring
- Audio routing state
- Speakerphone state

A normal background application should not assume unrestricted access to call state or audio. Android's Telecom framework is primarily designed around the system phone application and call services.

([developer.android.google.cn](https://developer.android.google.cn/develop/connectivity/telecom?hl=en&utm_source=openai))

The application should respond to:

```
```text
CALL_RINGING
CALL_ACTIVE
CALL_HOLD
CALL_ENDED
CALL_FAILED
MONITORING_STARTED
MONITORING_STOPPED
```
```

4.5 Monitoring Consent

Monitoring must never silently start.

Before audio capture:

- Show a clear user-facing confirmation.
- Explain that the microphone will be active.
- Explain whether audio is processed locally or uploaded.
- Show a persistent foreground-service notification.
- Provide a visible stop button.
- Stop monitoring immediately when the call ends.
- Record consent timestamp and consent version.

Example confirmation:

> Trust Voice will use your microphone while this call is active to analyze the conversation. Monitoring will stop when the call ends. Continue?

4.6 Speakerphone Control

The application may detect audio routing and, depending on call role and device support, request speakerphone routing. The user should still explicitly confirm speakerphone use.

Android exposes speaker routing through call audio state APIs, but support depends on the application's Telecom role and device implementation.

([developer.android.google.cn](https://developer.android.google.cn/reference/android/telecom/CallAudioState?authuser=50&utm_source=openai))

4.7 Live Audio Capture

For microphone capture:

- Use `AudioRecord`.
- Use `MediaRecorder.AudioSource.VOICE\_COMMUNICATION` where appropriate.
- Use 16 kHz mono PCM for speech analysis.
- Use 20 ms or 30 ms frames.
- Use a bounded in-memory ring buffer.
- Avoid writing raw audio to disk unless required.
- Apply acoustic echo cancellation where available.
- Apply noise suppression and automatic gain control carefully.

Recommended audio pipeline:

```
```text
```

Microphone

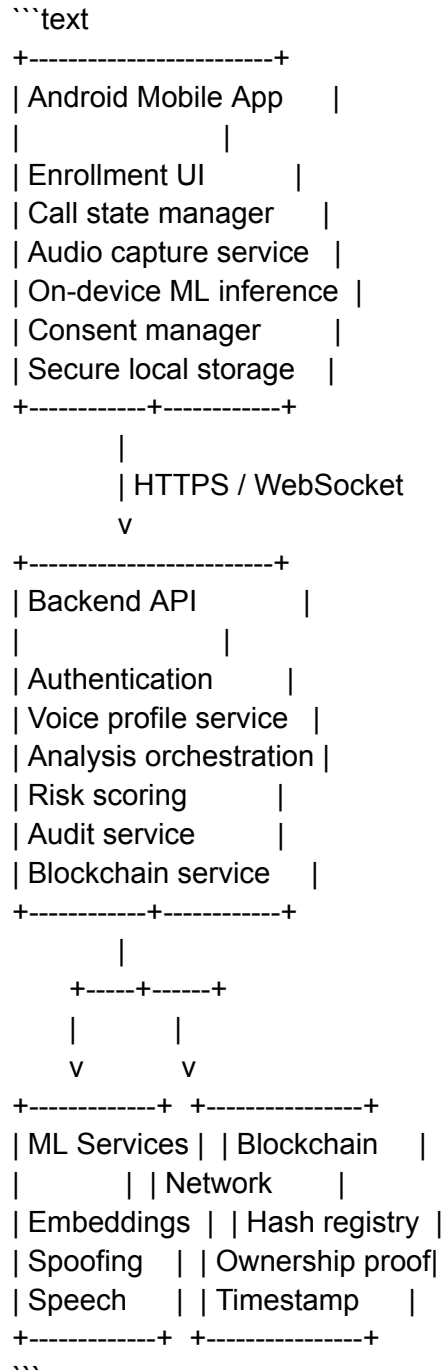
- > AudioRecord
- > Audio preprocessing
- > Voice activity detection
- > Echo/noise suppression
- > Feature extraction
- > Voice embedding model
- > Spoof detection model
- > Risk engine
- > UI alert/event storage

```
```
```

Android warns that multiple applications may compete for microphone access and that active audio capture behavior depends on priority and platform rules.

([developer.android.com](https://developer.android.com/media/platform/sharing-audio-input?hl=en&utm_source=openai))

5. System Architecture



6. Recommended Technology Stack

The exact PPT stack should be applied once it is available. Until then, the following stack is suitable.

Android

- Kotlin
- Jetpack Compose
- Android SDK
- Coroutines and Flow
- Hilt
- Retrofit or Ktor client
- Room
- DataStore
- Android Keystore
- `AudioRecord`
- Foreground Service
- WebRTC for VoIP mode
- ONNX Runtime Mobile or TensorFlow Lite for on-device ML

Backend

Recommended options:

- TypeScript with NestJS
- PostgreSQL
- Redis
- Object storage such as S3-compatible storage
- WebSocket gateway
- Docker
- Kubernetes or managed container platform

Alternative:

- Python FastAPI for ML-heavy services
- PostgreSQL
- Redis
- Celery or a queue system
- PyTorch model-serving layer

Blockchain

Use the blockchain only for verification metadata, not raw voice data.

Store:

```
```text
voice_profile_hash
user_public_key
profile_version
created_at
revocation_status
```
```

Do not store:

- Raw voice recordings.
- Voice embeddings.
- Phone numbers.
- Names.
- Call transcripts.
- Personally identifiable information.

Suitable implementation options:

- EVM-compatible chain.
- Permissioned enterprise chain.
- Layer-2 network for low transaction fees.
- Smart contract with profile registration and revocation.

7. Data Storage Model

Users

```
```sql
users (
 id UUID PRIMARY KEY,
 email TEXT,
 phone TEXT,
 status TEXT NOT NULL,
 created_at TIMESTAMP NOT NULL,
 updated_at TIMESTAMP NOT NULL
)
```
```

Voice Profiles

```
```sql
voice_profiles (
```

```
id UUID PRIMARY KEY,
user_id UUID NOT NULL,
model_version TEXT NOT NULL,
embedding_version TEXT NOT NULL,
profile_hash TEXT NOT NULL,
blockchain_tx_id TEXT,
quality_score NUMERIC,
status TEXT NOT NULL,
created_at TIMESTAMP NOT NULL,
revoked_at TIMESTAMP
)
...
```

### ### Monitoring Sessions

```
```sql  
monitoring_sessions (  
id UUID PRIMARY KEY,  
user_id UUID NOT NULL,  
call_type TEXT NOT NULL,  
started_at TIMESTAMP NOT NULL,  
ended_at TIMESTAMP,  
consent_version TEXT NOT NULL,  
audio_mode TEXT NOT NULL,  
result TEXT,  
risk_score NUMERIC  
)  
...
```

Risk Events

```
```sql  
risk_events (
id UUID PRIMARY KEY,
session_id UUID NOT NULL,
event_type TEXT NOT NULL,
severity TEXT NOT NULL,
confidence NUMERIC,
evidence_reference TEXT,
created_at TIMESTAMP NOT NULL
)
...
```

## ## 8. Blockchain Design

### ### Smart Contract Functions

```
```solidity
registerVoiceProfile(
    bytes32 profileHash,
    bytes32 profileVersion
)

revokeVoiceProfile(
    bytes32 profileHash
)

verifyVoiceProfile(
    bytes32 profileHash
) view returns (bool)

getProfileOwner(
    bytes32 profileHash
) view returns (address)
```
```

### ### Important Design Rule

Blockchain data is immutable. Therefore:

- Store only hashes and status references.
- Never place personal or biometric data directly on-chain.
- Use revocation records instead of attempting deletion.
- Use a user-controlled wallet or custodial wallet depending on product requirements.
- Support key rotation.
- Protect transaction submission through a backend signer or secure wallet service.

### ### Hash Process

```
```text
Encrypted voice profile
-> Canonical serialization
-> SHA-256 or Keccak-256
-> blockchain profile hash
```
```

The same profile should be reproducibly hashed only when the exact encrypted artifact is preserved. For easier verification, define a canonical profile manifest and hash that manifest.

## ## 9. Machine Learning Pipeline

### ### 9.1 Voice Verification

The voice verification model compares live audio against the enrolled voice profile.

Output:

```
```text
speaker_similarity_score
confidence
voice_match
```
```

The system must not treat voice matching as absolute identity proof. Voice changes with:

- Illness.
- Stress.
- Age.
- Background noise.
- Microphone quality.
- Emotional state.
- Codec compression.

### ### 9.2 Replay and Spoof Detection

Detect:

- Replayed recordings.
- Synthetic speech.
- Voice conversion.
- Deepfake audio.
- Edited segments.
- Abnormal spectral characteristics.
- Repeated audio patterns.
- Excessively clean or inconsistent audio.

### ### 9.3 Speech-to-Text

Use speech-to-text only when legally permitted and required.

Possible outputs:

```
```text
transcript
language
keywords
speaker_segments
confidence
```
```

The default design should process audio on-device where possible and upload only extracted features or risk events.

### ### 9.4 Fraud and Risk Engine

Risk factors may include:

- Voice mismatch.
- Synthetic speech probability.
- Replay probability.
- Urgency language.
- Requests for passwords or verification codes.
- Requests for money or cryptocurrency.
- Account takeover language.
- Remote-control instructions.
- Threats or coercion.
- Caller identity mismatch.

Example score:

```
```text
risk_score =
    0.35 * voice_mismatch +
    0.25 * spoof_probability +
    0.20 * suspicious_language +
    0.10 * replay_probability +
    0.10 * call_context_risk
```
```

Scores must be calibrated using real validation data. Do not expose a score as a definitive claim that a caller is fraudulent.

## ## 10. API Specification

### ### Authentication

```
```http
POST /v1/auth/register
POST /v1/auth/login
POST /v1/auth/refresh
POST /v1/auth/logout
```
```

### ### Voice Enrollment

```
```http
POST /v1/voice/enrollment/sessions
POST /v1/voice/enrollment/sessions/{id}/samples
POST /v1/voice/enrollment/sessions/{id}/complete
GET /v1/voice/profile
POST /v1/voice/profile/revoke
```
```

### ### Monitoring

```
```http
POST /v1/monitoring/sessions
POST /v1/monitoring/sessions/{id}/events
POST /v1/monitoring/sessions/{id}/complete
GET /v1/monitoring/sessions/{id}/result
```
```

### ### Blockchain Verification

```
```http
POST /v1/blockchain/voice-profiles/register
GET /v1/blockchain/voice-profiles/{hash}
POST /v1/blockchain/voice-profiles/{hash}/revoke
```
```

### ### Live Analysis WebSocket

```
```text
wss://api.trustvoice.example/v1/live-analysis/{session_id}
```
```

### Messages:

```
```json
{
```

```
"type": "audio_features",
"timestamp_ms": 1700000000,
"features": "base64-encoded-feature-data"
}
...
```

Server response:

```
```json
{
 "type": "risk_update",
 "risk_score": 0.78,
 "voice_match": 0.42,
 "spoof_probability": 0.81,
 "severity": "high",
 "reason_codes": [
 "VOICE_MISMATCH",
 "POSSIBLE_SYNTHETIC_AUDIO"
]
}
...
```

Avoid sending raw audio over the network by default. If raw audio is necessary, encrypt the transport, limit retention, and document the purpose.

## ## 11. Android Application Modules

```
```text
app/
  auth/
  enrollment/
  voiceprofile/
  call/
  monitoring/
  audio/
  ml/
  risk/
  blockchain/
  consent/
  security/
  storage/
  networking/
...
```

Suggested classes:

```text

CallStateManager  
MonitoringForegroundService  
AudioCaptureManager  
AudioPreprocessor  
VoiceEmbeddingEngine  
SpoofDetectionEngine  
RiskScoringEngine  
ConsentManager  
VoiceProfileRepository  
BlockchainRepository  
MonitoringSessionRepository  
```

12. Foreground Service Requirements

The monitoring service should:

- Start only after explicit user action.
- Display a persistent notification.
- Use microphone foreground-service classification where required by Android version.
- Stop when the call ends.
- Handle microphone denial and route changes.
- Recover from temporary audio focus loss.
- Avoid unrestricted background recording.
- Log only operational metadata.

The application should target current Android requirements and test foreground microphone behavior across Android 12 through the current supported release.

13. Privacy and Security Requirements

Encryption

- TLS 1.3 for network connections.
- AES-256-GCM for encrypted profile data.
- Android Keystore for local key protection.
- Envelope encryption for server-side storage.
- Separate encryption keys per user or profile.
- No plaintext biometric data in logs.

Access Control

- Users can access only their own profile.
- Administrators cannot directly view raw voice data by default.
- ML services receive the minimum data required.
- Use short-lived signed URLs for object storage.
- Audit every profile access.

Privacy Controls

Users must be able to:

- View stored data.
- Revoke monitoring consent.
- Delete off-chain data.
- Revoke voice profile status.
- Export account data.
- Disable cloud analysis.
- Disable blockchain registration, if product policy permits.

Blockchain records cannot be physically deleted, so this must be clearly explained before registration.

14. Legal and Compliance Requirements

This product processes biometric data and potentially call content. Legal review is mandatory before launch.

Required workstreams:

- Biometric privacy laws.
- State call-recording and interception laws.
- Consent requirements for all participants.
- GDPR or equivalent requirements if applicable.
- CCPA/CPRA if applicable.
- Data retention policy.
- Data deletion policy.
- App store privacy declarations.
- Security incident response.
- Law-enforcement request process.

The app must not secretly record calls. Monitoring must be visible and user-controlled. For any transcript or server-side analysis, define explicit consent and retention rules.

15. Threat Model

Threats

- Replay of the user's recorded voice.
- Deepfake or synthetic voice.
- Theft of encrypted profile.
- Compromise of device.
- Blockchain key theft.
- Malicious application attempting microphone access.
- False positive fraud alert.
- False negative voice match.
- Network interception.
- Insider access to biometric data.
- Call participant recording without consent.

Controls

- Randomized enrollment phrases.
- Liveness and anti-replay detection.
- Device attestation where appropriate.
- Secure key storage.
- Encryption at rest and in transit.
- No raw voice logging.
- Rate limits.
- Transaction signing controls.
- Risk-score calibration.
- Human-readable explanations for alerts.
- Model monitoring and drift detection.

16. Testing Strategy

Unit Tests

- Call state transitions.
- Consent state machine.
- Audio frame buffering.
- Profile hashing.
- Encryption/decryption.
- Risk score calculations.
- Blockchain transaction handling.

Android Tests

- Microphone permission denied.

- Call starts while app is closed.
- Call ends unexpectedly.
- Speakerphone toggled.
- Bluetooth connected.
- Headphones connected.
- Screen locked.
- Battery saver enabled.
- Another app uses the microphone.
- Audio focus lost.
- Android 12, 13, 14, 15, and current supported versions.
- Multiple OEMs including Samsung, Pixel, and OnePlus.

ML Tests

- Same-speaker verification.
- Different-speaker verification.
- Replay attacks.
- Synthetic voices.
- Noise and echo.
- Accents and languages.
- Illness and stress.
- Low-volume recordings.
- Codec-compressed audio.
- Speakerphone audio.

Security Tests

- Static analysis.
- Dependency scanning.
- API authorization tests.
- Token replay tests.
- Key extraction attempts.
- Storage inspection.
- TLS interception resistance.
- Blockchain transaction authorization.
- Penetration testing.

Performance Targets

Recommended initial targets:

```text

Voice alert latency: less than 2 seconds

Audio frame size: 20-30 ms

Enrollment completion: less than 3 minutes  
On-device inference CPU: less than 25% average  
Battery use during monitoring: less than 8% per hour  
Crash-free sessions: greater than 99.5%  
...

These must be validated on real devices, not only emulators. Android documentation notes that the emulator is not suitable for microphone recording validation.  
([developer.android.com](https://developer.android.com/media/platform/mediarecorder?utm\_source=openai))

## ## 17. Development Phases

### ### Phase 0: Feasibility and Legal Validation

Deliverables:

- Android call-audio feasibility matrix.
- Supported-device list.
- Consent design.
- Legal review.
- Decision between cellular monitoring and VoIP-first architecture.

### ### Phase 1: Product Foundation

Deliverables:

- Authentication.
- User profile.
- Consent management.
- Secure local storage.
- Backend skeleton.
- Database.
- CI/CD pipeline.

### ### Phase 2: Voice Enrollment

Deliverables:

- Guided enrollment.
- Audio quality checks.
- Voice embedding generation.
- Encrypted profile storage.
- Profile versioning.

- Profile revocation.

#### ### Phase 3: Blockchain Registry

Deliverables:

- Smart contract.
- Profile hash registration.
- Transaction monitoring.
- Verification endpoint.
- Key management.
- Revocation flow.

#### ### Phase 4: Speakerphone Monitoring MVP

Deliverables:

- Call-state detection.
- Monitoring foreground service.
- Microphone capture.
- Speakerphone workflow.
- Local audio preprocessing.
- Basic voice matching.
- Risk alerts.

#### ### Phase 5: Live Backend Analysis

Deliverables:

- WebSocket streaming.
- Feature extraction service.
- Spoof detection.
- Speech-to-text where approved.
- Risk engine.
- Post-call report.

#### ### Phase 6: VoIP Calling

Deliverables:

- In-app calling.
- WebRTC audio pipeline.
- Two-channel separation.
- Reliable live analysis.

- Call quality monitoring.
- Network recovery.

### ### Phase 7: Production Hardening

#### Deliverables:

- Penetration testing.
- Privacy audit.
- Model validation.
- OEM compatibility testing.
- Play Store compliance.
- Monitoring and incident response.
- Production release.

### ## 18. Acceptance Criteria

#### The MVP is complete when:

- A user can create an account.
- A user can enroll a voice profile using multiple samples.
- Voice data is encrypted.
- Raw samples are not stored unless explicitly enabled.
- A profile hash can be verified on-chain.
- A user can revoke a profile.
- Monitoring requires explicit consent.
- A foreground notification is visible during monitoring.
- Monitoring stops when the call ends.
- Speakerphone audio can be analyzed on supported devices.
- The app clearly reports when audio capture is unavailable.
- Risk events are shown with confidence and explanations.
- No raw audio or biometric data appears in logs.
- Security and privacy tests pass.
- The supported-device and unsupported-device behavior is documented.

### ## 19. Key Product Decision

The team should not promise “Trust Voice can capture and analyze every cellular call” using a normal Android application. The technically defensible product statement is:

> Trust Voice creates a cryptographically registered voice identity and analyzes supported live audio sessions. Standard phone calls can be monitored only when Android device routing and user-approved microphone access permit speakerphone capture. For reliable two-way analysis, Trust Voice provides an in-app VoIP calling mode.

That distinction should be reflected in the product requirements, marketing, architecture, legal documentation, and acceptance tests.